

C05-AT-3(ASSESSMENT TOOL 15)

AAMEENA I(192511284)

Q1)

Interviewer: Explain the steps of query processing in DBMS. How does a query optimizer decide the best execution plan?

Answer:

Query processing is the sequence of steps a DBMS follows to translate a high-level SQL query into an efficient low-level execution strategy that retrieves the required data from disk. It consists of four main phases:

1. Parsing and Translation

The SQL query is checked for syntactic and semantic correctness by the parser, then translated into an internal representation such as a relational algebra expression or a query tree/graph.

2. Query Optimization

The optimizer examines multiple equivalent relational algebra expressions (obtained through transformation rules) and multiple possible execution plans (different join orders, join algorithms, and access paths), then estimates the cost of each using statistics such as table size, index availability, and selectivity of predicates.

3. Evaluation Plan Selection and Code Generation

The plan with the least estimated cost (in terms of disk I/O, CPU time, and memory usage) is chosen, and low-level code or an internal query evaluation plan is generated for it.

4. Execution

The query evaluation engine executes the chosen plan and returns the result to the user.

How the optimizer decides the best plan:

- It generates several candidate execution plans by trying different join orders, join methods (nested loop, hash join, merge join) and access paths (full scan vs index scan).
- It uses cost estimation formulas based on catalog statistics (number of tuples, number of distinct values, index heights) to predict the number of disk I/Os and CPU cycles for each plan.
- It compares the total estimated cost of all candidate plans and picks the one with the minimum cost, which becomes the actual execution plan handed to the executor.

Q2)

**Interviewer: What is the difference between heuristic optimization and cost-based optimization?
When would you use each?**

Answer:

Both are strategies used inside the query optimizer to improve performance, but they differ in approach:

Heuristic (Rule-Based) Optimization

- Applies fixed transformation rules to the query tree regardless of the actual data, e.g., 'perform selection and projection as early as possible', 'combine cartesian products with following selections into joins', 'process the most restrictive selection first'.
- Does not use table statistics or cost estimates — it simply assumes certain rewrites are always beneficial.
- Fast to apply since no cost computation is needed, but may not always produce the truly optimal plan.

Cost-Based Optimization

- Generates multiple candidate execution plans and estimates the actual cost (I/O, CPU, memory) of each using statistics stored in the system catalog.
- Selects the plan with the lowest estimated cost, so it adapts to the real size and distribution of the data.
- More accurate but computationally expensive, since it must evaluate many plan alternatives.

When to use each:

- Heuristic optimization is preferred for simple queries or systems with limited statistics, where speed of optimization matters more than finding the absolute best plan.
- Cost-based optimization is preferred for complex queries with multiple joins on large tables, where the cost difference between a good and a bad plan is significant — most modern commercial DBMS (Oracle, SQL Server, PostgreSQL) use cost-based optimization as the primary strategy, often after first applying heuristic rewrites to reduce the search space.

Q3)

Interviewer: How would you implement database connectivity in a real project? Explain the difference between JDBC and ODBC.

Answer:

In a real project, database connectivity is implemented by using a driver/API layer that lets the application code communicate with the DBMS without dealing with vendor-specific network protocols directly. A typical implementation involves: (1) adding the appropriate driver library to the project, (2) establishing a connection using a connection string (host, port, database name, credentials), (3) creating a statement or prepared statement object, (4) executing SQL queries/updates, (5) processing the result set, and (6) closing the connection/releasing resources (ideally using connection pooling for performance).

JDBC (Java Database Connectivity)

- A Java-based API that allows Java applications to connect to a database using vendor-specific JDBC drivers.
- Platform-independent — the same Java code works across operating systems as long as a JDBC driver exists for the target database.
- Object-oriented, tightly integrated with the Java language and exception handling model.

ODBC (Open Database Connectivity)

- A language-independent C-based API/standard developed by Microsoft that allows any application to connect to any database that has an ODBC driver.
- Platform-dependent — typically tied closely to Windows, though drivers exist for other platforms.
- Acts as a universal translator between an application and different DBMS vendors using standardized function calls.

In short: JDBC is designed specifically for Java and is fully object-oriented, while ODBC is a more general, language-neutral standard usable from C/C++ and other languages; in fact, the JDBC-ODBC bridge historically allowed Java applications to use existing ODBC drivers.

Q4)

Interviewer: What are ACID properties? Give a real-world example where each property is critical.

Answer:

ACID properties guarantee that database transactions are processed reliably:

Atomicity

A transaction is treated as a single indivisible unit — either all its operations complete, or none do.

Example: In a bank fund transfer, if money is debited from Account A but the system crashes before crediting Account B, atomicity ensures the entire transaction is rolled back so A is not left short.

Consistency

A transaction moves the database from one valid state to another, preserving all defined rules and constraints. Example: In an airline reservation system, the total number of booked seats can never exceed the aircraft's capacity — consistency ensures this constraint always holds after a booking transaction.

Isolation

Concurrent transactions execute as if they were run one after another, without interfering with each other's intermediate results. Example: If two customers try to book the last available movie ticket at the same time, isolation ensures only one transaction succeeds and the other sees the updated (sold-out) state.

Durability

Once a transaction is committed, its changes persist even in the event of a system crash or power failure.

Example: After an ATM withdrawal is confirmed and the receipt is printed, the debited balance must remain permanently recorded even if the bank's server crashes moments later.

Q5)

Interviewer: Explain concurrency control problems: dirty read, lost update, and unrepeatable read. How does 2PL solve them?

Answer:

Dirty Read

Occurs when a transaction reads data written by another transaction that has not yet committed. If the writing transaction later rolls back, the reading transaction has used invalid/nonexistent data.

Lost Update

Occurs when two transactions read the same data and both update it based on the value they read; the second write overwrites the first, so the first transaction's update is silently lost.

Unrepeatable Read

Occurs when a transaction reads the same row twice and gets different values because another committed transaction modified the row in between the two reads.

How Two-Phase Locking (2PL) solves these problems

2PL requires every transaction to acquire all the locks it needs before releasing any lock, dividing its execution into a growing phase (acquiring locks) and a shrinking phase (releasing locks, no new locks acquired afterward).

- Dirty reads are prevented because a transaction must hold a shared lock to read data, which cannot be granted while another transaction holds an exclusive (write) lock on the same item until it commits.
- Lost updates are prevented because a transaction must hold an exclusive lock to write data, so a second transaction cannot write to the same item until the first releases its lock, forcing serialized updates.
- Unrepeatable reads are prevented because holding a lock for the full duration of the transaction (as in Strict 2PL) blocks other transactions from modifying data that has already been read, ensuring repeated reads return the same value.

Overall, 2PL guarantees conflict-serializable schedules, which eliminates these anomalies, though it can introduce deadlocks that must be handled separately.

Q6)

Interviewer: What is deadlock in DBMS? Describe the deadlock detection algorithm and at least two prevention strategies.

Answer:

A deadlock is a state in which two or more transactions are each waiting for a lock held by the other, so none of them can proceed — forming a cycle of dependencies with no possible resolution without external intervention.

Deadlock Detection Algorithm

The DBMS periodically constructs a Wait-For Graph (WFG), where each transaction is a node and an edge from T_i to T_j means T_i is waiting for a lock held by T_j . The system checks this graph for cycles; if a cycle exists, a deadlock has occurred. The DBMS then selects a victim transaction (typically based on criteria like least amount of work done, fewest locks held, or rollback cost) and aborts it, releasing its locks so the remaining transactions can proceed.

Prevention Strategies

- Wait-Die scheme: If an older transaction requests a lock held by a younger transaction, the older transaction is allowed to wait; if a younger transaction requests a lock held by an older transaction, the younger transaction is aborted (dies) and restarted later.
- Wound-Wait scheme: If an older transaction requests a lock held by a younger transaction, it forces (wounds) the younger transaction to abort and release the lock; if a younger transaction requests a lock held by an older one, it waits.
- Lock ordering: Impose a global ordering on all data items and require every transaction to request locks strictly in that order, which makes a circular wait (and hence a deadlock) structurally impossible.
- Timeout-based prevention: A transaction waiting longer than a predefined threshold is automatically aborted and restarted, avoiding indefinite waiting without needing to build a wait-for graph.

Q7)

Interviewer: Compare Immediate Update and Deferred Update recovery. In what scenarios is each preferred?

Answer:

Immediate Update

Changes made by a transaction are written to the database on disk as soon as they occur, even before the transaction commits. To support recovery, the DBMS must maintain both UNDO and REDO information in the log, since uncommitted changes may already be on disk.

- On recovery: committed transactions are REDONE (to make sure their effects are reflected) and uncommitted/incomplete transactions are UNDONE (to remove their partial effects).

Deferred Update

Changes made by a transaction are held in a buffer/log and are only written to the database once the transaction commits. Because uncommitted data never touches the actual database, only REDO information is needed — there is nothing to undo since uncommitted work never reached the disk.

- On recovery: only committed transactions need to be REDONE using the log; there is no UNDO phase.

When each is preferred

- Immediate Update is preferred in systems where buffer memory is limited and changes must be flushed to disk early for performance/space reasons, and where the overhead of maintaining detailed undo logs is acceptable.
- Deferred Update is preferred in systems where simplicity of recovery is important and enough buffer space is available to hold all changes until commit, since it avoids the complexity and overhead of undo operations entirely.

Q8)

Interviewer: Explain Shadow Paging. What are its advantages and limitations compared to log-based recovery?

Answer:

Shadow Paging is a recovery technique that avoids logs altogether by maintaining two page tables: a current page table, which points to the most recent (in-progress) versions of database pages, and a shadow page table, which points to the pages as they existed before the transaction started and is never modified during the transaction. When a transaction writes to a page, a new copy of that page is created and the current page table is updated to point to it, while the shadow page table continues to point to the original unmodified page. If the transaction commits, the shadow page table is simply replaced with the current page table (an atomic pointer switch). If the transaction aborts or the system crashes before commit, the current page table is discarded and the shadow page table (still pointing to the original consistent state) becomes the database state again.

Advantages

- No need for UNDO/REDO logging — recovery is nearly instantaneous since it just involves discarding the current page table or committing it.
- Recovery is very fast because there is no need to replay a log.

Limitations

- Data fragmentation occurs over time because pages are copied rather than updated in place, scattering related data across disk.
- Garbage collection is required to reclaim old shadow pages that are no longer referenced, adding overhead.
- Difficult to support concurrent transactions efficiently, since each transaction may need its own page table copy, and commit is not easily extended to multiple simultaneous transactions.

- Not well suited for large databases with high transaction volumes, which is why log-based recovery (with UNDO/REDO logs) is far more common in practice.

Q9)

Interviewer: What is a serializability schedule? How do you test for conflict serializability using a precedence graph?

Answer:

A serializable schedule is a schedule of concurrently executing transactions whose final result is equivalent to the result of running those same transactions one at a time in some serial order. Serializability is the correctness criterion used to allow transactions to run concurrently (for performance) while still guaranteeing the same outcome as a serial (one-at-a-time) execution.

Testing Conflict Serializability using a Precedence Graph

A precedence graph (also called a serialization graph) is constructed as follows:

- Create one node for every transaction in the schedule.
- For every pair of conflicting operations (two operations on the same data item, from different transactions, where at least one is a write) where an operation of T_i occurs before the conflicting operation of T_j , draw a directed edge from T_i to T_j .
- After considering all conflicting pairs, examine the resulting graph.

If the precedence graph contains no cycles, the schedule is conflict-serializable, and a valid serial order can be obtained by performing a topological sort of the graph. If the graph contains a cycle, the schedule is not conflict-serializable, because the cycle represents contradictory ordering requirements that cannot be satisfied by any single serial execution.

Q10)

Interviewer: Describe the Two-Phase Locking protocol variations: Basic 2PL, Strict 2PL, and Rigorous 2PL. Which is most commonly used and why?

Answer:

Basic 2PL

Every transaction has a growing phase, in which it can acquire locks but not release any, and a shrinking phase, in which it can release locks but not acquire any new ones. This guarantees conflict-serializability but locks may be released before commit, which can lead to cascading rollbacks if a transaction aborts after releasing a lock that another transaction has already used.

Strict 2PL

Adds the rule that all exclusive (write) locks held by a transaction are released only after the transaction commits or aborts. Shared (read) locks may still be released earlier. This prevents other transactions from reading uncommitted (dirty) data, eliminating cascading rollbacks caused by dirty reads.

Rigorous 2PL

The strictest variation: both shared and exclusive locks are held until the transaction commits or aborts — no locks are released early at all. This provides the simplest and safest recovery behavior, since a transaction's locked data is never visible or modifiable by others until it is fully finished.

Most Commonly Used

Strict 2PL is the most widely used variation in real-world database systems because it strikes a practical balance: it prevents cascading rollbacks and dirty reads (unlike Basic 2PL) while still allowing shared locks to be released a little earlier than Rigorous 2PL, giving slightly better concurrency. Most commercial DBMS engines implement locking behavior equivalent to Strict 2PL as their default isolation mechanism.
